certora

Security Assessment

Final Report

Spectra Bridge

May 2026

Prepared for Spectra

certora

Table of Contents

certora

Project Summary

Project Scope

Project Name	Repository Link	Commit Hash	Platform
Spectra Bridge	https://github.com/perspectivefi/audit-bridge-stellar https://github.com/perspectivefi/spectra-contracts-configs	fed17f1 (initial) 729e914 (latest) 01ea96d (latest)	Stellar, EVM

Project Overview

This document describes the security review of **Spectra Bridge**. The work was undertaken from **May 1st, 2026** to **May 18th, 2026**.

The following contract list is included in our scope:

```
audit-bridge-stellar/*
spectra-contracts-configs/*
```

The team performed a manual audit of all the files in scope**.** During the manual audit, the Certora team discovered bugs in the code, as listed on the following page.

Protocol Overview

The Spectra Stellar ecosystem is a hybrid DeFi protocol that extends Spectra's Principal Token functionality to the Stellar network through a symmetric EVM↔Stellar bridge and a deterministic Soroban-based oracle system. It employs a modular Diamond Router framework to enable complex, batch-executed transactions involving multi-asset swaps, flashloans, and cross-chain bridging across both EVM and Soroban environments. By integrating secure messaging via Axelar and LayerZero with advanced on-chain math for price discovery, the project provides a unified

infrastructure for bridging and trading interest-bearing asset derivatives. This comprehensive stack ensures secure, rate-limited asset mobility.

Assessment Methodology

Our assessment approach combines design level analysis with a deep review of the implementation to ensure that a protocol is secure, economically sound, and behaves as intended under realistic conditions.

At the design level, we evaluate the architecture, the economic assumptions behind the protocol, and the safety properties that should hold independently of a specific chain or environment. This process includes reviewing internal and cross protocol interactions, state transition flows, trust boundaries, and any mechanism that could be exploited to extract value, deny service, or alter core system behavior. At this stage, a focused threat modelling exercise helps identify key attack surfaces and adversarial capabilities relevant to the system. Design level issues often relate to incentive structures, governance implications, or systemic behavior that emerges under adversarial conditions.

Implementation analysis focuses on the concrete behavior of the code within the execution model of the target chain. This involves reviewing the correctness of logic, access control, state handling, arithmetic behavior, and the nuanced behaviors of the chain environment. Familiar classes of vulnerabilities such as reentrancy conditions, faulty permission checks, precision issues, or unsafe assumptions often surface at this layer. These findings require context aware reasoning that takes into account both the code and the architectural intent.

To support this analysis, the codebase is examined through repeated manual passes and supplemented by automated tools when appropriate. High-risk logic areas receive deeper scrutiny, invariants are validated against both design intent and actual implementation, and potential vulnerability leads are thoroughly investigated. Automated techniques such as static analysis, fuzzing, or symbolic execution may be used to complement manual review and provide additional insight.

Collaboration with the development team plays an important role throughout the audit. This helps confirm expected behaviors, clarify design assumptions, and ensure an accurate understanding of the protocol's intended operation. All findings are documented with clear reasoning, reproducible examples, and actionable recommendations. A follow up review is conducted to validate the applied fixes and verify that no regressions or secondary issues have been introduced.

Threat and Security Overview

1. Core Security Architecture

The ecosystem employs a defense-in-depth strategy across its hybrid infrastructure:

- **Modular Diamond Framework**: The EVM Router uses a selector-based dispatch system to isolate execution logic. This modularity allows for granular updates but shifts the security burden to the **Command-Dispatch Trust Surface**, where the router must ensure zero-balance state transitions to prevent fund spillage.
- **Symmetric Cross-Chain Trust**: The Bridge anchors its security in a dual-verification model. Inbound messages must not only pass gateway-level authentication (Axelar/LayerZero) but also match a strictly defined validation, ensuring that only authorized source contracts can trigger minting or unlocking events.
- **Granular Access Control**: Security is managed through a multi-tiered role system. On EVM, this is centralized via OpenZeppelin's `AccessManager`, while the Soroban side utilizes a decentralized 4-role model to minimize the impact of single-role compromises.

2. Expected Invariants

The properties intends to uphold across bridge, wrapped PT, oracles, and the diamond router.

- **Bridge inbound gate (Stellar + EVM)**: Inbound mint/unlock runs only if the caller is the allowed messenger (current ± previous where applicable), the source address matches the configured trusted remote for that chain, and the bridge is not paused.
- **Wrapped PT integrity**: Wrapped supply changes only through bridge-authorized mint/burn.
- **Accounting**: Fee helpers split gross into fee + net (`fee + net == gross`); ledgered fees match what can be withdrawn and unlock/mint amounts match the checked message.
- **Cross-chain replay:** Per-message exactly-once is enforced by the messenger/gateway layer; the bridge contracts enforce **who** may deliver and **policy** on payload, not global ordering or liveness of the network.

• **Oracles:** Sensitive oracle parameters change only via owner/upgrader-gated paths; Integrators reading price / lastprice see outputs fully determined by current on-chain parameters.

• **Diamond router:** Permissionless `execute` for users; **any** change to implementation pointers, command table, or privileged config goes through AccessManager-`restricted` / `manageFunctions` / `manageCommands`; when paused, the diamond `fallback` does not run user facets.

3. Primary Threat Vectors

• **Arbitrary Execution & Reentrancy**: The Router's flexibility allows for complex batching, including re-entrant flashloans. The primary risk involves "Confused Deputy" attacks where malicious user-supplied wrappers or pools could exploit the router's approvals to sweep accumulated dust or native ETH. Other covered cases include arbitrary user-supplied address risks, unauthorized execution of router calls, correctness of external protocol implementations, stale cached batch parameters, malicious module protection, initialization correctness.

• **Cross-Chain Desynchronization**: The integrity of the bridge relies on the invariant that `EVM-Locked == Stellar-Minted`. Threats to this include rate-limit window desynchronization, failing messages, payload metadata poisoning, bridge provider blocking, messenger swaps not impacting inbound messages or skewing the bridge.

• **Stellar-Specific Risks**: On the Soroban side, the protocol faces unique challenges such as TTL management. Without active "bumping" of contract and storage TTLs, critical infrastructure like the Axelar Messenger could be archived, leading to permanent cross-chain DoS.

• **Oracle Manipulation**: While the Spectra Oracles use deterministic ZCB math, the administrative ability to set `future_pt_value` introduces a centralized point of failure. If compromised, this could allow for instant price shocks, potentially triggering cascading liquidations in downstream protocols.

4. Security Mechanisms & Mitigations

• **Rate Limiting**: Both sides implement signed-int128 volume counters to cap the economic impact of any single message-layer breach, providing a critical buffer against massive drainage.

• **Pause & Circuit Breakers**: Granular pause controls are implemented in both the Diamond Router and the `PTBridge`, allowing operators to freeze specific execution paths (like `fallback`) or the entire bridge during suspected exploits.

• **TTL & Persistence Hygiene**: To combat Soroban's storage rent model, the bridge implements extensive TTL extensions on all hot read/write paths, ensuring that mapping data and contract instances remain active.

• **Fallback messenger**: A tracking mechanism for the previous allowed messenger to let inbound messages pass correctly in case there is an update to the existing messenger or the underlying cross-chain protocol changes

certora

Findings Summary

The table below summarizes the findings of the review, including type and severity details.

Severity	Discovered	Confirmed	Fixed
Critical	-	-	-
High	-	-	-
Medium	4	4	4
Low	6	6	5
Informational	3	3	1
Total	**13**	**13**	**10**

Severity Matrix

Impact	High	Medium	High	Critical
	Medium	Low	Medium	High
	Low	Low	Low	Medium
		Low	Medium	High
			Likelihood	

Detailed Findings

ID	Title	Severity	Status
M-01	Stranded gas tokens inside the messenger	Medium	Fixed
M-02	Messenger protocol changing can leave stranded inbound messages	Medium	Fixed
M-03	Mutable bridge fees can cause users to pay higher fees than expected	Medium	Fixed
M-04	Inconsistent rate limit handling between EVM and Soroban can permanently strand Inbound stellar messages	Medium	Fixed
L-01	Rate-limit window gets reduced by the entire pre-fee gross amount of tokens	Low	Acknowledged
L-02	Inconsistent usage of transaction value	Low	Fixed
L-03	Caller is always the refund receiver	Low	Fixed
L-04	Deterministic oracle deployment can be front-run by an attacker causing DoS	Low	Fixed
L-05	Missing TTL extension for persistent storage DataKey entries can lead to DoS.	Low	Fixed

L-06	Bridge messages with zero address recipient can cause permanent loss of funds	Low	Fixed
I-01	Exposed asymmetric burn functionality	Informational	Acknowledged
I-02	Oracle decimals serve only readability purposes	Informational	Acknowledged
I-03	allowInitializePath does not validate remote sender	Informational	Fixed

Medium Severity Issues

M-01: Stranded gas tokens inside the messenger

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: spectra-bridge-stellar/contracts/stellar/contracts/bridge-axelar/src/lib.rs	Status: Fixed	

Description:

In the Soroban AxelarMessenger contract, the send_message function pays for cross-chain gas using the Axelar Gas Service. It transfers gas_amount from the sender to itself, then calls pay_gas.

```
gas_service_client.pay_gas(

    &env.current_contract_address(),    // sender

    &destination_chain,

    &destination_address,

    &payload,

    &env.current_contract_address(),    // spender (refund excess here)

    &Token { address: gas_token, amount: gas_amount },

    &Bytes::new(&env),                  // empty metadata

    )
```

The spender argument is set to env.current_contract_address(). However, the messenger has no function to withdraw tokens or forward refunds back to the original user. Thus any excess gas paid by the user for a cross-chain transaction on Stellar is permanently stranded.

Recommendations:

Refund the gas tokens to the sender of the message and not the messenger used

Customer Response:

Fixed in c261aac.

Fix Review:

Fix confirmed.

M-02: Messenger protocol changing can leave stranded inbound messages

Severity: Medium	Impact: Medium	Likelihood: Medium
Files: spectra-bridge-stellar\contracts\evm\src\adapters\AxelarMessenger.sol spectra-bridge-stellar\contracts\evm\src\adapters\LayerZeroMessenger.sol	Status: Fixed	

Description:

The bridge is designed to support messenger rotation through previousMessenger, but the trusted remote configuration is not messenger-aware. PTBridge stores a single trustedRemotes[sourceChain] string and compares every inbound message against that single value, regardless of whether the message was delivered by the current messenger or the previous messenger.

This breaks migrations between Axelar and LayerZero because the adapters forward different source address formats to the bridge:

- `AxelarMessenger` forwards the gateway-provided `sourceAddress` string unchanged.
- `LayerZeroMessenger` converts `Origin.sender` into a `0x`-prefixed 32-byte hex string.

During a messenger switch, the operator must update trustedRemotes[sourceChain] to the new messenger's source format. Once that happens, valid in-flight messages delivered by previousMessenger can still pass the messenger authorization check, but then fail the trusted remote check because their source address uses the old messenger's format. In-flight cross-chain messages can become permanently unexecutable during Axelar <-> LayerZero migration. If a user has already locked or burned assets on the source chain, the corresponding mint or unlock on the destination chain can revert, leaving the user's transfer stranded.**Failure Scenario:**

1. The bridge is using Axelar.

2. `trustedRemotes["stellar-2026-q1-2"]` is configured to the Stellar Axelar messenger source address, for example a Stellar strkey-style address.

3. A user initiates a valid Stellar -> EVM transfer.

4. The source-chain side burns or locks the user's asset and sends an Axelar message.

5. Before the Axelar message is delivered on EVM, the operator switches the bridge to LayerZero by calling `setMessenger(layerZeroMessenger)`.

6. The old Axelar messenger is now stored in `previousMessenger`.

7. To make new LayerZero messages pass, the operator updates `trustedRemotes["stellar-2026-q1-2"]` to the LayerZero peer format, a `0x`-prefixed 32-byte hex string.

8. The old Axelar message is delivered.

9. The message passes `onlyMessenger` because it came through `previousMessenger`.
10. The message then fails `trustedRemotes[sourceChain] == sourceAddress` because the stored trusted remote is now the LayerZero hex peer, while the Axelar message still carries the Axelar source string.

11. The destination execution reverts with `UntrustedSource`.

If the operator keeps the Axelar trusted remote instead, new LayerZero messages fail for the same reason. The current design cannot accept both source formats for the same chain during a migration.

Recommendations:

Make trusted remotes messenger-awarep via either allowing multiple trusted remotes per chain, or having separate trusted remote configurations depending on who the messenger is.

Customer Response:

Fixed in 02381bd.

Fix Review:

Fix confirmed.

M-03: Mutable bridge fees can cause users to pay higher fees than expected

Severity: Medium	Impact: High	Likelihood: Low
Files: spectra-bridge-stellar/contracts/evm/src/PTBridge.sol spectra-bridge-stellar/contracts/stellar/contracts/bridge/src/lib.rs	Status: Fixed	

Description:

Bridge fee rates are mutable and can be updated by a privileged role. The fee charged to users is determined at execution time rather than when the transaction is quoted or signed.

As a result, users may prepare a bridge transaction assuming a lower fee, but a fee update included before their transaction execution can cause them to pay a higher fee than expected.

Additionally, transaction reorgs can change whether the fee update or the user's bridge transaction executes first. As a result, the transaction may execute either before or after the fee change compared to what the user originally observed, causing the final charged fee to differ from the quoted or expected value.

Because the bridge functions do not include a user-specified maximum acceptable fee, users cannot enforce a fee cap at execution time.

Recommendations:

Consider adding a user-specified maximum acceptable fee and revert if the configured fee at execution exceeds that value.

Customer Response:

Fixed in 59eef3b.

Fix Review:

Fix confirmed.

M-04: Inconsistent rate limit handling between EVM and Soroban can permanently strand Inbound stellar messages

Severity: Medium	Impact: High	Likelihood: Low
Files: spectra-bridge-stellar/contracts/stellar/contracts/bridge/src/lib.rs	Status: Fixed	

Description:

There is an inconsistency in how rate limits are handled between the EVM and Soroban implementations.

On the EVM side, a per-token limit of 0 or an unset limit is treated as “no limit,” causing the function to return without applying any rate-limiting checks.

However, on the Soroban side, when the per-token limit is unset, the logic falls back to the global volume rate limit instead of disabling rate limiting entirely. Additionally, setting the limit to 0 is not allowed on the Soroban side. Per the comment it is intended to disable the limit globally as well, but it is never enforced via the code.

Because of this mismatch, a user may successfully bridge an amount from the EVM side that exceeds the Stellar global limit, since no restriction is enforced on the source chain.

When the corresponding inbound message is later processed on Stellar/Soroban, the receive transaction will fail due to the global volume rate limit being exceeded or set to 0, causing a revert instead of a `return`.

As a result, inbound Stellar messages can become temporarily stranded because the transfer was accepted on the source chain but couldn’t be successfully executed on the destination chain under the enforced global limit conditions.

Recommendations:

Ensure both the EVM and Soroban implementations handle unset or 0 per-token limits consistently. Either both implementations should treat an unset or 0 value as 'no limit' or both

should fall back to the global rate limit. Additionally, allow explicitly setting the per-token limit to 0 on the Stellar side.

Customer Response:

Fixed in b516c60, 729e914, and aad97ef.

Fix Review:

Fix confirmed.

Low Severity Issues

L-01: Rate-limit window gets reduced by the pre-fee gross amount of tokens

Severity: Low	Impact: Low	Likelihood: Medium
Files: spectra-bridge-stellar\contracts\evm\src\PTBridge.sol	Status: Acknowledged	

Description:

The rate limiting logic on the bridge consumes the full gross amount of assets before deducting the fee. This is a design choice, but it also misleads and cripples the real token amounts that can travel back and forth for a given window:

- Outbound (EVM): rate limit consumes `amount` (gross); locked balance grows by `bridgeAmount = amount - feeAmount`.
- Inbound (EVM): rate limit restores `msg_.amount` = `burn_amount` (net after Stellar fee).
- Fee tokens exit via `withdrawFees` which have no rate limit check.

Each fee-paying round trip permanently consumes feeAmount of rate limit window budget even though the net economic flow is zero. With a 10% fee and limit=1000, after one round trip the window sits at -100. A second 1000-token outbound pushes to -1100 and reverts, even though the user's net position is unchanged causing the 2 rate limits to desync from each other.

Recommendations:

Consider letting only the net amount account towards the actual bridge limit.

Customer Response:

Acknowledged.

L-02: Inconsistent usage of transaction value

Severity: Low	Impact: Low	Likelihood: Medium
Files: diamond-router\src\modules\CommandsModule.sol	Status: Fixed	

Description:

The router executes a sequence of commands atomically. In commands that require native ETH, the modules utilize the cached LibExecutionModule.getMsgValue() to determine how much ETH to forward. Because getMsgValue() relies on the transaction's original msg.value which is cached at the beginning of the batch, a user trying to chain multiple BRIDGE_PT_TO_STELLAR or KYBER_SWAP commands will fail their entire batch transactions, even if it can succeed with the remaining value from previous commands.The value of the message is also reset on nested flashloan calls due to address(this).call not passing along the current value.

Recommendations:

Consider recalculating the remaining actual msg.value of the batch after it is passed for an external call.

Customer Response:

Fixed in d0c8f36 and aad97ef.

Fix Review:

Fix confirmed.

L-03: Caller is always the refund receiver

Severity: Low	Impact: Medium	Likelihood: Low
Files: spectra-bridge-stellar/contracts/evm/src/PTBridge.sol spectra-bridge-stellar/contracts/stellar/contracts/bridge/src/lib.rs	Status: Fixed	

Description:

Aftermath of M-01, when a user bridges tokens from the source chain to the destination chain, the default behavior should always set the refund address to the caller, and the gas token is used for fees.

If a smart contract interacts with the bridge, the smart contract itself becomes the refund receiver. This can create issues if the contract does not implement gas handling functionality, it may become incompatible and will not be able to accept the refund later.

Recommendations:

Consider adding a refundReceiver parameter to the bridge functions, or clearly documenting this behavior.

Customer Response:

Fixed in c261aac.

Fix Review:

Fix confirmed.

L-04: Deterministic oracle deployment can be front-run by an attacker causing DoS

Severity: Low	Impact: Low	Likelihood: Low
Files: spectra-oracles-stellar/contracts/spectra-oracle-factory/src/lib.rs	Status: Fixed	

Description:

deploy_oracle is permissionless and allows anyone to deploy an oracle contract using a caller provided salt. Since the resulting contract address is deterministically derived from the (factory, salt, wasm_hash) combination, the same deployment cannot be executed twice.

An attacker who guesses or observes the victim's salt can front-run the transaction, use the victim's salt, and submit their own deployment first. Once the attacker's transaction is confirmed, the victim's original deployment will permanently fail because the contract for that salt has already been created, resulting in a DoS for the intended oracle deployment. Additionally, anyone can deploy an oracle for any PT address with arbitrary parameters. Since there is no binding between PTs and their intended oracles, it relies on the assumption that the correct oracles will be tracked, which can lead to confusion.

Recommendations:

Generate the deployment salt on-chain using the oracle parameters.

Alternatively, restrict `deploy_oracle` to an authorized role if public deployment is not required by the protocol.

Customer Response:

Fixed in 6f8f14b.

Fix Review:

Fix confirmed.

L-05: Missing TTL extension for persistent storage DataKey entries can lead to DoS.

Severity: Low	Impact: Medium	Likelihood: Low
Files: spectra-bridge-stellar/contracts/stellar/contracts/bridge/src/lib.rs	Status: Fixed	

Description:

For data types RateLimitPerPt, ChainName, and PREV_MSGR, the TTL is extended when they are initially set in persistent storage, but it is never extended again when the entries are later accessed or used. Additionally, for the AccumulatedFees data type, the value is stored in persistent storage but its TTL is never extended at all, causing it to rely solely on the minimum_ttl.

When the TTL of a persistent entry expires, the entry becomes archived and must be restored before it can be used again.

If a transaction attempts to access an archived persistent storage entry, it lead to a DoS, because archived persistent entries cannot be recreated or accessed directly. They must first be restored before they can be modified or deleted.

Recommendations:

Extend the TTL for RateLimitPerPt, ChainName, PREV_MSGR, and AccumulatedFees whenever they are used, not just when they are initially set.

Customer Response:

Fixed in 210ee9f.

Fix Review:

Fix confirmed.

L-06: Bridge messages with zero address recipient can cause permanent loss of funds

Severity: Low	Impact: Low	Likelihood: Low
Files: spectra-bridge-stellar/contracts/stellar/contracts/bridge/src/lib.rs	Status: Fixed	

Description:

In the bridge_back_to_evm and bridge_native_pt_to_evm functions, there is no validation to ensure that the recipient address is not the zero address.

As a result, a bridge message can be created with the recipient set to the zero address.

Execution of this message will perpetually fail because every attempt to execute the receiveMessage function will revert, as the OpenZeppelin ERC20 implementation does not support minting or transferring tokens to the zero address.

As a result, the tokens on the Stellar chain may remain permanently locked or burned, with the message stuck indefinitely.

Recommendations:

Validate in the bridge_back_to_evm and bridge_native_pt_to_evm function that the recipient address is not the zero address.

Customer Response:

Fixed in d5230be.

Fix Review:

Fix confirmed.

Informational Severity Issues

I-01: Exposed asymmetric burn functionality

Files:

spectra-bridge-stellar/contracts/stellar/contracts/wrapped-pt/src/lib.rs

Description:

The Stellar WrappedPT contract exposes burn() and burn_from(). However, on the EVM side WrappedStellarPT.burn() is correctly restricted to onlyBridge, preventing this class of desync. The Stellar side has no equivalent restriction on the standard SEP-41 burn path. Wrapped PTs can be arbitrarily burned.

Recommendations:

Consider restricting the calls to these functions, they serve no user purpose, but potential leaking allowances can be used to burn user tokens

Customer Response:

Acknowledged.

I-02: Oracle decimals serve only readability purposes

Files:

spectra-oracles-stellar\contracts\spectra-oracle\src\lib.rs

Description:

Oracle decimals are purely for metadata, as the price returned price on each query is never validated nor normalized against the provided decimals. The future_pt_value is blindly assumed to follow them correctly, which raises risk.

Recommendations:

Consider implementing a sanity check, a normalization or documenting this behavior if it is to remain.

Customer Response:

Acknowledged.

I-03: allowInitializePath does not validate remote sender

Files:

spectra-bridge-stellar/contracts/evm/src/adapters/LayerZeroMessenger.sol

Description:

allowInitializePath only checks whether a peer exists for the source endpoint ID and does not validate that the remote sender matches the configured trusted peer:

```
function allowInitializePath(Origin calldata _origin) external view returns
(bool) {
    return peers[_origin.srcEid] != bytes32(0);
}
```

When a DVN verifies a message for a pathway the first time, it calls allowInitializePath on the receiver to check if messages from that sender and source chain are allowed.

As a result, any sender from a source chain with a configured peer can initialize the pathway during DVN verification, even if the sender is not the trusted remote application.

While lzReceive correctly validates the sender, allowInitializePath behaves inconsistently with the standard LayerZero OApp security model and may allow unintended path initialization.

Recommendations:

Validate both the source endpoint ID and the remote sender address:

```
return peers[_origin.srcEid] == _origin.sender;
```

This ensures that only the configured trusted peer can initialize the pathway.

Customer Response:

Fixed in 016bc62.

Fix Review:

Fix confirmed.

Disclaimer

Even though we hope this information is helpful, we provide no warranty of any kind, explicit or implied. The contents of this report should not be construed as a complete guarantee that the contract is secure in all dimensions. In no event shall Certora or any of its employees be liable for any claim, damages, or other liability, whether in an action of contract, tort, or otherwise, arising from, out of, or in connection with the results reported here.

About Certora

Certora is a Web3 security company that provides industry-leading formal verification tools and smart contract audits. Certora's flagship security product, Certora Prover, is a unique SaaS product that automatically locates even the most rare & hard-to-find bugs on your smart contracts or mathematically proves their absence. The Certora Prover plugs into your standard deployment pipeline and is widely used by developers and security researchers during audits and bug bounties.

Certora also provides architectural design reviews, where researchers analyze system design and trust boundaries early to identify structural risks and reduce complexity before implementation.

Certora conducts off-chain audits covering backend services, APIs, frontend, and mobile applications, focusing on vulnerabilities in authentication, data handling, key management, and wallet-related workflows.

Certora also offers Penetration Testing and Red Teaming to simulate real-world attacks and uncover exploitable weaknesses across infrastructure, applications, and business logic.

In addition, Certora provides operational security (OpSec) assessments, reviewing key management, access controls, and operational processes to strengthen overall security posture.

Finally, Certora delivers ongoing monitoring and incident response, enabling continuous threat detection and prevention, alert triage, and coordinated response to active incidents.